Product

We have a Utilization Dashboard product. It shows how well faculty and rooms are used etc, datasource attached. Next to it we need to showcase a Utilization Assistant, a chat window where a dean or administrator asks questions in plain language about faculty workload, room use and timetables in plain language and get an answer in seconds, without waiting for a report. Known questions run as fixed, reviewed templates so answers are consistent and fast.

Few key terms in the project for reference -

- Faculty workload: weekly load against the mandated hours for each academic rank. Load is timetable teaching hours plus PhD supervision and experiential hours. The mandate is 4 hours for a School Dean, 10 for a Professor, 12 for an Associate Professor, 14 for an Assistant Professor and 8 for a full-time PhD Scholar, less any load relaxation (2 hours under current policy). Supervision counts 0.25 hours per week per PhD Scholar.
- Room utilization: Room Busy % (scheduled hours out of 45 a week, which is 9 periods over 5 days) and Seat Occupancy % (enrollment against seating capacity).
- Faculty-student ratio: students per faculty member at school level. Enrolments are only totalled for each school; LIA holds no student-level data and no breakdown by programme or semester.
- Timetable schedule: the weekly schedule for any room, faculty member or section.

Data

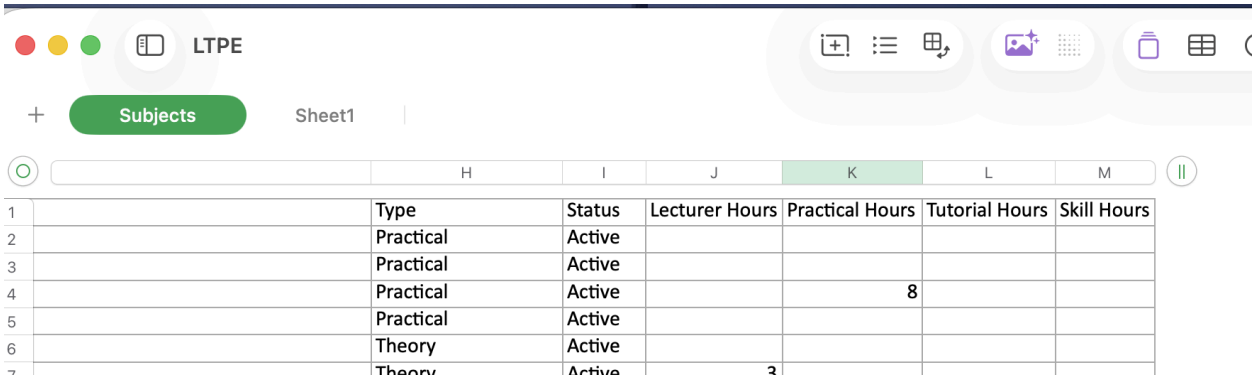
Attached are 6 Excel files exported from Postgress DB.
https://drive.google.com/drive/folders/1pIFJWd0PXAozBrEbjlu-g3qxDkjxofF7?usp=sharing

The timetable is a single weekly schedule, free of clashes, that repeats every week. A day has 9 periods, Monday to Friday. A practical takes two consecutive rows. Not every staff member teaches, and rooms are shared across schools, so state your assumptions.

Data source	Fields available
Timetable	Day, period name, from and to time, location (room), subject code and name, credits, staff code and name, section, semester, programme, department (school), degree (UG or PG), session type (Lecture, Practical, Tutorial), class type, number of enrolments, timetable start and end date
Faculty Master	Staff ID, name, designation, staff category, gender, home department (school), active status
Room Master	School, block, floor, room number, seating capacity, remarks (for example Lab), room description (for example Mac Lab, Circuits Lab)

Course LTPE mapping	Department, course code, course name, credits, lecture (L), tutorial (T), practical (P) and experiential (E) hours
PhD supervision	Faculty member, PhD scholars mapped, scholar type (full-time or part-time), supervision hours
Load relaxation	Faculty member, school, relaxation hours, justification

* In course LTPE mapping - hours are on a weekly basis.



	H	I	J	K	L	M
1	Type	Status	Lecturer Hours	Practical Hours	Tutorial Hours	Skill Hours
2	Practical	Active				
3	Practical	Active				
4	Practical	Active		8		
5	Practical	Active				
6	Theory	Active				
7	Theory	Active	3			

What LIA is and how it should behave

Requirement	What LIA does
Asks when a question is unclear	Asks one short question, with touch options, when a question can be read in more than one way and the answers would differ. Asks at most two questions before answering. Example: "Show occupancy for last month" gets "Room utilisation or faculty teaching load?"
Touch first, typing sparingly	Suggested questions, clarifying choices, parameters (period, school, threshold) and follow-up questions are all touch options. A typing box is always available, but used sparingly.
Autocomplete from question templates	As the user types, LIA suggests matching question templates (section 4), grouped by area, with editable slots for a number, period, school or name. A chosen template runs a fixed, reviewed query, so known questions need no model-written query. Typed text with no match is answered dynamically, labelled for owner review, and frequent questions are approved and added as templates.
Shows how it calculated	Every answer has a "How I got this" section, closed by default with its first line visible and a drop-down arrow. Opened, it shows the filters, period, exclusions, assumptions and data as-of time.

Suggests a next best option when data is missing	If the requested period has not been uploaded, LIA offers the nearest period with data as a labelled proxy, or an email to the data owner asking for the upload. It never substitutes a period silently and sends no email without confirmation.
Read-only and role-aware	LIA never changes CAMU or the timetable. A Dean sees only their own school; university-level users see all schools.

Chat Agent should support below :

Build a working Text-to-SQL chatbot prototype with a React frontend, Python backend, and actual LLM integrations supporting both Google Gemini and local Ollama models.

Consider a modular, agentic architecture (Langgraph etc) incorporating LLM routing, preprocessing, SQL validation, evaluation (Langfuse), and other relevant components. Keep schema retrieval modular and add vector search (Qdrant) to retrieve relevant tables, columns, business definitions, and example question-to-SQL pairs for complex schemas.

Please review the requirements, identify any missing aspects, and incorporate additional details or technical considerations needed to build a robust, functional, and extensible Text-to-SQL chatbot prototype.

Project will be evaluated on -
Agentic orchestration, schema linking, metadata retrieval, Text-to-SQL, semantic layer, conversational memory, LLM routing, SQL guardrails, self-correction loop, execution-grounded validation, evaluation framework, observability, fallback strategy, and RAG.

Provide clear instructions to run and verify the prototype locally. **Ensure the chat agent handles below chat queries** :

1- **Use cases and sample question templates**: This list is not exhaustive, and we will also be evaluated on unseen queries related to the domain.

https://docs.google.com/spreadsheets/d/1WdCQx6-8bPaMUgHrfYDbkv3RRoA4dY1vGYdIp_j_ydg/edit?usp=sharing

2- Handling questions that return a large result

3- Handling out-of-scope questions